Lập trình mạng · Chương 5

Hiệu năng: lời gọi không chặn với CompletableFuture và gRPC async stub

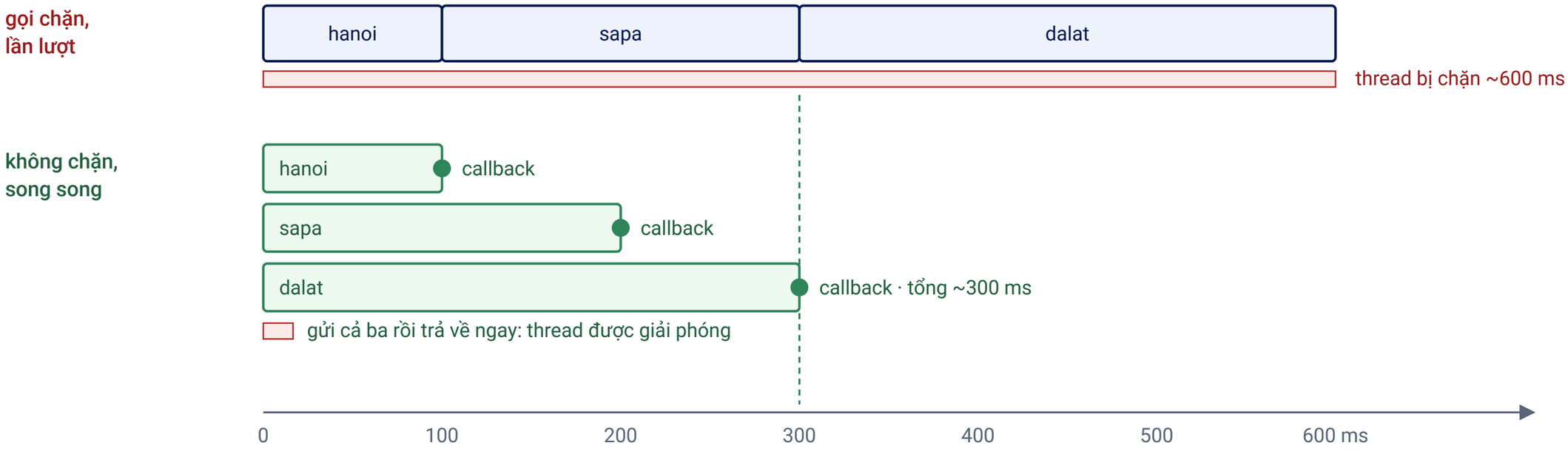
TS. Đặng Ngọc Hùng

Ba lời gọi nối đuôi nhau, thread chỉ chờ

1. Dữ liệu chia trên **ba server**; một yêu cầu tổng hợp cần cả ba khóa
2. **Blocking stub** gọi lần lượt: thời gian là **tổng** ba lời gọi, thread chỉ chờ
3. Một server lỗi làm hỏng cả yêu cầu

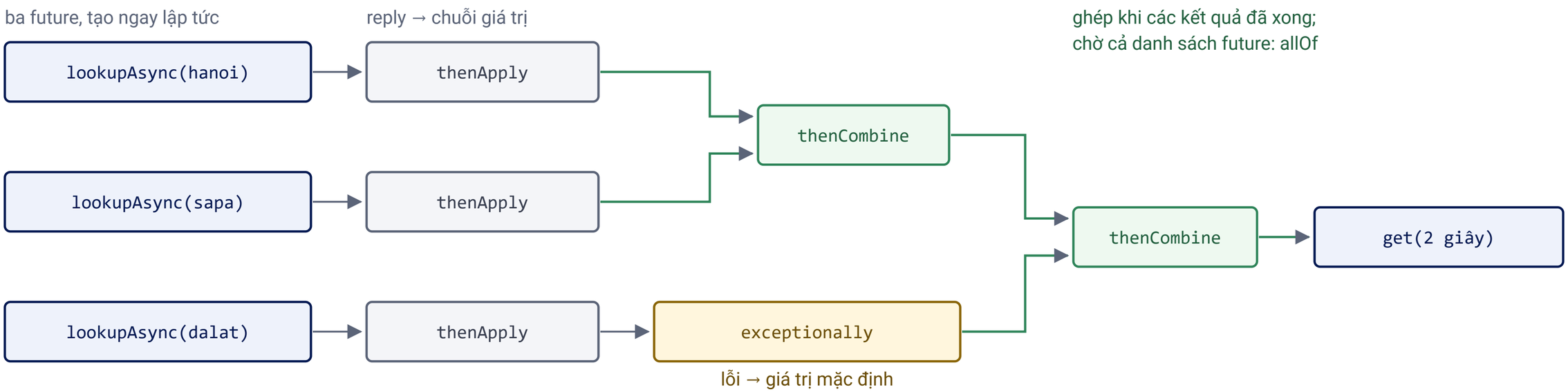
> **Câu hỏi của phần:** gửi cả ba lời gọi mà không chờ, rồi ghép kết quả và xử lý lỗi của từng lời gọi thế nào?

Lời gọi không chặn: thời gian chờ bằng lời gọi chậm nhất



Lời gọi không chặn trả về ngay, reply được xử lý qua **callback**; thời gian chờ chung bằng lời gọi **chậm nhất**.

CompletableFuture: nối bước xử lý, thay lỗi, ghép kết quả



CompletableFuture: `thenApply` biến đổi kết quả, `exceptionally` thay lỗi, `thenCombine` và `allOf` ghép kết quả; thread chỉ chờ một lần, ở `get`.

Async stub: gRPC gọi StreamObserver khi reply tới, callback hoàn tất future

```java
CompletableFuture<LookupReply> future = new CompletableFuture<>();
LookupGrpc.newStub(channel).lookup(LookupRequest.newBuilder().setKey(key).build(), new StreamObserver<>() {
    public void onNext(LookupReply reply) {
        future.complete(reply);  // callback: chạy trên thread của gRPC khi reply tới
    public void onError(Throwable t) {
        future.completeExceptionally(t);
return future;
```

- `newStub` tạo **async stub**: lời gọi trả về ngay sau khi gửi request
- gRPC gọi `onNext` hay `onError` trên thread của thư viện khi reply hoặc lỗi tới
- Callback chỉ hoàn tất future: phía gọi làm việc với **CompletableFuture**

Đo thời gian: gọi lần lượt và gọi song song ba server

Java

```
CompletableFuture<String> hanoi = Lookups.lookupAsync(hanoiShard, "hanoi").thenApply(LookupReply::getValue);
CompletableFuture<String> sapa = Lookups.lookupAsync(sapaShard, "sapa").thenApply(LookupReply::getValue);
CompletableFuture<String> dalat = Lookups.lookupAsync(dalatShard, "dalat").thenApply(LookupReply::getValue)
String combined = hanoi.thenCombine(sapa, (a, b) -> a + " | " + b)  // ghép hai kết quả khi cả hai xong
        .thenCombine(dalat, (ab, c) -> ab + " | " + c)
        .get(2, TimeUnit.SECONDS);
```

Kết quả chạy

```
blocking  hanoi -> Thủ đô của Việt Nam
blocking  sapa -> Thị trấn vùng cao
blocking  dalat -> Thành phố ngàn hoa
blocking  total 671 ms
async     Thủ đô của Việt Nam | Thị trấn vùng cao | Thành phố ngàn hoa
async     total 307 ms
```

Một server lỗi: exceptionally thay lỗi bằng giá trị mặc định

Java

```
CompletableFuture<String> dalat = Lookups.lookupAsync(dalatShard, "dalat").thenApply(LookupReply::getValue)
        .exceptionally(t -> "(không tra được: " + Status.fromThrowable(t).getCode() + ")");  // lỗi → mặc định
```

Kết quả chạy

```
blocking  hanoi -> Thủ đô của Việt Nam
blocking  sapa -> Thị trấn vùng cao
blocking  failed: UNAVAILABLE
blocking  total 335 ms
async     Thủ đô của Việt Nam | Thị trấn vùng cao | (không tra được: UNAVAILABLE)
async     total 214 ms
```

Callback chạy trên thread của thư viện: thao tác chặn chuyển sang executor riêng

Java

```
ExecutorService callbackThread = Executors.newSingleThreadExecutor(r -> new Thread(r, "grpc-callback"));
all[i] = ownExecutor
        ? reply.thenApplyAsync(CallbackDemo::slowSave, work)  // thao tác chặn chạy trên executor riêng
        : reply.thenApply(CallbackDemo::slowSave);            // thao tác chặn chạy ngay trên thread callback
```

Kết quả chạy

```
blocking work in callback       total 1653 ms
blocking work on own executor   total 807 ms
```

- Thao tác chặn trong callback giữ thread của thư viện: reply tới sau phải xếp hàng
- Tác vụ nặng, I/O chặn: chuyển sang **executor riêng** bằng `thenApplyAsync`

Câu hỏi 1

Ba lời gọi không chặn tới ba server lần lượt mất 100 ms, 200 ms và 300 ms, rồi được ghép bằng thenCombine. Tổng thời gian chờ khoảng bao nhiêu?

A. 100 ms, bằng lời gọi nhanh nhất.

B. 200 ms, bằng thời gian trung bình.

C. 600 ms, bằng tổng ba lời gọi.

D. 300 ms, bằng lời gọi chậm nhất.

Câu hỏi 1

Ba lời gọi không chặn tới ba server lần lượt mất 100 ms, 200 ms và 300 ms, rồi được ghép bằng thenCombine. Tổng thời gian chờ khoảng bao nhiêu?

A. 100 ms, bằng lời gọi nhanh nhất.

B. 200 ms, bằng thời gian trung bình.

C. 600 ms, bằng tổng ba lời gọi.

D. 300 ms, bằng lời gọi chậm nhất.

Ba lời gọi được gửi gần như cùng lúc và chờ song song; kết quả ghép chỉ có khi lời gọi chậm nhất xong. 600 ms là thời gian của cách gọi lần lượt.

Câu hỏi 2

Thư viện chạy mọi callback trên một thread duy nhất. Mỗi callback ghi kết quả xuống đĩa, mất 500 ms. Ba reply tới gần như cùng lúc: điều gì xảy ra?

A. Reply thứ hai và thứ ba bị mất.

B. Thư viện tạo thêm thread để chạy song song các callback.

C. Các callback xếp hàng trên thread đó, tổng thời gian khoảng 1,5 giây.

D. Không ảnh hưởng gì, vì lời gọi đã là không chặn.

Câu hỏi 2

Thư viện chạy mọi callback trên một thread duy nhất. Mỗi callback ghi kết quả xuống đĩa, mất 500 ms. Ba reply tới gần như cùng lúc: điều gì xảy ra?

A. Reply thứ hai và thứ ba bị mất.

B. Thư viện tạo thêm thread để chạy song song các callback.

C. Các callback xếp hàng trên thread đó, tổng thời gian khoảng 1,5 giây.

D. Không ảnh hưởng gì, vì lời gọi đã là không chặn.

Thread callback đang bận với thao tác chặn thì các reply khác chưa được xử lý; ba thao tác 500 ms nối nhau. Lời gọi không chặn chỉ giải phóng thread gọi, không làm callback hết chặn.

Tổng kết

1. **Giao tiếp bất đồng bộ**: gửi lời gọi rồi trả về ngay; thời gian chờ bằng lời gọi chậm nhất

2. **Async stub** báo reply qua **callback**; **CompletableFuture** nối bước, thay lỗi, ghép kết quả

3. Callback chạy trên thread của thư viện: thao tác chặn chuyển sang executor riêng

Từ khóa cần nhớ CompletableFuture · async stub · callback · giao tiếp bất đồng bộ

Mở rộng

Chủ đề nâng cao

- Reactive Streams và back-pressure
- Virtual thread như lựa chọn thay cho mã bất đồng bộ

Bài tập mở rộng

Viết client tra cứu 100 khóa bằng async stub, giới hạn tối đa 10 lời gọi đang chạy.

Tiếp theo

Hiệu năng: NIO, Selector và mẫu Reactor